

~ -- -zsh

... |

~ -- -zsh

~ -- nano executor.c

+

UW PICO 5.09File: executor.cModified

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64
#define BUF_SIZE 512

// 1. ESCAPE SEQUENCE PARSER: Treats '\' as an escape character[cite: 14]
int parse_escaped_input(const char *input, char *args[]) {
    int arg_count = 0;
    int len = strlen(input);
    int i = 0;

    while (i < len) {
        // Skip leading whitespace[cite: 14]
        while (i < len && isspace((unsigned char)input[i])) i++;
        if (i >= len) break;

        char buf[BUF_SIZE];
        int buf_idx = 0;

        while (i < len && !isspace((unsigned char)input[i])) {
            // Handle Backslash Escape Sequences (\ , \[, \>, etc.)[cite: 14]
            if (input[i] == '\\' && i + 1 < len) {
                i++; // Skip backslash[cite: 14]
                buf[buf_idx++] = input[i++]; // Store escaped character literally[cite: 14]
            } else {
                buf[buf_idx++] = input[i++];
            }
        }

        buf[buf_idx] = '\0';
        args[arg_count++] = strdup(buf);
    }

    args[arg_count] = NULL; // Null-terminate argument array for execvp[cite: 15]
    return arg_count;
}

// 2. PROCESS LAUNCHER: Spawns child processes using fork() and execvp()[cite: 15]
void execute_command(char *args[]) {
    if (args[0] == NULL) return; // Empty command[cite: 15]

    pid_t pid = fork();
    if (pid < 0) {
        // Get Help
        // Exit
        // WriteOut
        // Justify
        // Read File
        // Where is
        // Prev Pg
        // Next Pg
        // Cut Text
        // UnCut Text
        // Cur Pos
        // To Spell
```

~ -- -zsh

... |

~ -- -zsh

~ -- nano executor.c

+

UW PICO 5.09File: executor.cModified

```
        while (i < len && !isspace((unsigned char)input[i])) {
            // Handle Backslash Escape Sequences (\ , \[, \>, etc.)[cite: 14]
            if (input[i] == '\\' && i + 1 < len) {
                i++; // Skip backslash[cite: 14]
                buf[buf_idx++] = input[i++]; // Store escaped character literally[cite: 14]
            } else {
                buf[buf_idx++] = input[i++];
            }
        }

        buf[buf_idx] = '\0';
        args[arg_count++] = strdup(buf);
    }

    args[arg_count] = NULL; // Null-terminate argument array for execvp[cite: 15]
    return arg_count;
}

// 2. PROCESS LAUNCHER: Spawns child processes using fork() and execvp()[cite: 15]
void execute_command(char *args[]) {
    if (args[0] == NULL) return; // Empty command[cite: 15]

    pid_t pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        return;
    } else if (pid == 0) {
        // Child Process: Execute program[cite: 15]
        if (execvp(args[0], args) == -1) {
            perror("Execution error");
            exit(EXIT_FAILURE);
        }
    } else {
        // Parent Process: Wait for child to complete[cite: 15]
        int status;
        waitpid(pid, &status, 0);
    }
}

// Memory Cleanup[cite: 15]
void free_args(char *args[], int count) {
    for (int i = 0; i < count; i++) {
        free(args[i]);
        args[i] = NULL;
    }
}

int main() {
    // Get Help
    // Exit
    // WriteOut
    // Justify
    // Read File
    // Where is
    // Prev Pg
    // Next Pg
    // Cut Text
    // UnCut Text
    // Cur Pos
    // To Spell
```

-- -zsh...-- -zsh-- nano executor.c

UW PICO 5.09File: executor.cModified

```
        perror("Fork failed");
        return;
    } else if (pid == 0) {
        // Child Process: Execute program[cite: 15]
        if (execvp(args[0], args) == -1) {
            perror("Execution error");
            exit(EXIT_FAILURE);
        }
    } else {
        // Parent Process: Wait for child to complete[cite: 15]
        int status;
        waitpid(pid, &status, 0);
    }
}

// Memory Cleanup[cite: 15]
void free_args(char *args[], int count) {
    for (int i = 0; i < count; i++) {
        free(args[i]);
        args[i] = NULL;
    }
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nExecutor Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Remove newline[cite: 16]
        if (strcmp(input_buf, "exit") == 0) break;

        char *args[MAX_ARGS];
        int arg_count = parse_escaped_input(input_buf, args);

        if (arg_count > 0) {
            printf("---- Debug: Parsed Command Arguments ----\n");
            for (int i = 0; i < arg_count; i++) {
                printf("Arg [%d]: \"%s\"\n", i, args[i]);
            }

            printf("---- Execution Output ----\n");
            execute_command(args);
            free_args(args, arg_count);
        }
    }
}


```

⌘ Get Help
⌘ Exit

⌘ WriteOut
⌘ Justify

⌘ Read File
⌘ Where is

⌘ Prev Pg
⌘ Next Pg

⌘ Cut Text
⌘ UnCut Text

⌘ Cur Pos
⌘ To Spell

-- -zsh...-- -zsh-- nano executor.c

UW PICO 5.09File: executor.cModified

```
char input_buf[BUF_SIZE];

while (1) {
    printf("\nExecutor Shell > ");
    if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

    input_buf[strcspn(input_buf, "\n")] = 0; // Remove newline[cite: 16]
    if (strcmp(input_buf, "exit") == 0) break;

    char *args[MAX_ARGS];
    int arg_count = parse_escaped_input(input_buf, args);

    if (arg_count > 0) {
        printf("---- Debug: Parsed Command Arguments ----\n");
        for (int i = 0; i < arg_count; i++) {
            printf("Arg [%d]: \"%s\"\n", i, args[i]);
        }

        printf("---- Execution Output ----\n");
        execute_command(args);
        free_args(args, arg_count);
    }
}

return 0;
}


```

⌘ Get Help
⌘ Exit

⌘ WriteOut
⌘ Justify

⌘ Read File
⌘ Where is

⌘ Prev Pg
⌘ Next Pg

⌘ Cut Text
⌘ UnCut Text

⌘ Cur Pos
⌘ To Spell

~ — -zsh

...

~ — -zsh

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano executor.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall executor.c -o executor
./executor
```

```
Executor Shell > echo Hello\ \| World
--- Debug: Parsed Command Arguments ---
Arg [0]: "echo"
Arg [1]: "Hello \| "
Arg [2]: "World"
--- Execution Output ---
Hello \| World
```

```
Executor Shell > invalidcmd123
--- Debug: Parsed Command Arguments ---
Arg [0]: "invalidcmd123"
--- Execution Output ---
Execution error: No such file or directory
```

```
Executor Shell > exit
medikondarakesh@Medikondas-MacBook-Air ~ %
```